

Graph Neural Networks and Gradient-Boosted Hybrid Stacking for Coarse-Grained Molecular Force Field Parameterization

Shaurya Srivastava

Department of Electrical Engineering, Indian Institute of Technology Kanpur

Course: EE798R – Pattern Recognition

GitHub Repository: <https://github.com/shaurya244/EE798-CG-ForceField-GNN>

Abstract—Determining bonded interaction parameters ($k_{\text{bond}}, r_0, k_{\text{angle}}, \theta_0$) for coarse-grained (CG) molecular force fields—predominantly MARTINI 3—conventionally requires weeks of computationally demanding all-atom molecular dynamics (MD) simulations and Iterative Boltzmann Inversion (IBI). In this work, we formulate bonded parameterization as an inductive pattern recognition task over molecular graphs and present an end-to-end framework combining Edge-Conditioned Message-Passing Graph Neural Networks (MPNN) with a Stage-2 Gradient-Boosted Tree (XGBoost) stacking architecture. The GNN backbone encodes multi-hop chemical topologies into continuous permutation-invariant latent representations, while the gradient-boosted decision forest resolves the fundamental continuous-to-discrete representational bottleneck arising from empirical force-field look-up tables. Trained and evaluated on a rigorously deduplicated dataset of 1,226 MARTINI 3 biomolecules (7,606 bonds and 4,630 angles), our SOTA Hybrid Model achieves a linear R^2 of **0.9398** ($\text{MAE} = 1,880.15 \text{ kJ/mol/nm}^2$) on bond stiffness and **0.9211** ($\text{MAE} = 12.42 \text{ kJ/mol/rad}^2$, $\text{MedAE} = 0.50 \text{ kJ/mol/rad}^2$) on angle stiffness, slashing angle MAE by 46.2% over pure deep learning baselines. Physical verification demonstrates 100.0% Velocity Verlet integration stability at $\Delta t = 20 \text{ fs}$ across all 122 held-out test molecules, with a median conformational Boltzmann overlap of $BC = 0.9948$ ($D_{KL} = 0.052 k_B T$). The complete pipeline executes forward parameterization in $< 2.1 \text{ ms}$ per molecule and automatically outputs syntax-verified GROMACS topologies.

Index Terms—Pattern Recognition, Graph Neural Networks, Message Passing, Coarse-Grained Molecular Dynamics, Hybrid Stacking, Force Field Parameterization, MARTINI 3, GROMACS.



1 INTRODUCTION AND MOTIVATION

MOLECULAR dynamics (MD) simulations represent a foundational computational tool across structural biology, materials science, and biophysics. By numerically integrating Newton’s second law of motion:

$$\mathbf{F}_i = m_i \frac{d^2 \mathbf{r}_i}{dt^2} = -\nabla_{\mathbf{r}_i} V(\mathbf{r}_1, \dots, \mathbf{r}_N), \quad (1)$$

MD enables the atomic-level observation of molecular processes such as drug-membrane permeation, viral capsid self-assembly, and polymer crystallization [1], [2]. However, all-atom (AA) simulations are severely constrained by femtosecond integration steps ($\Delta t \approx 1\text{--}2 \text{ fs}$), rendering microsecond-to-millisecond biological phenomena computationally intractable on modern supercomputers.

1.1 The Coarse-Graining Paradigm & The Parameterization Bottleneck

Coarse-grained (CG) models—most notably the **MARTINI 3** force field [1]—overcome this spatiotemporal horizon by clustering approximately four heavy atoms into single interaction beads. This bead mapping reduces the degrees of freedom by nearly an order of magnitude and smoothens the underlying free energy surface, allowing time steps an order of magnitude larger ($\Delta t = 20\text{--}40 \text{ fs}$).

Despite these computational gains, generating the empirical bonded potential functions:

$$V_{\text{bond}}(r) = \frac{1}{2} k_{\text{bond}} (r - r_0)^2, \quad (2)$$

$$V_{\text{angle}}(\theta) = \frac{1}{2} k_{\text{angle}} (\theta - \theta_0)^2, \quad (3)$$

presents a severe operational bottleneck. Traditionally, determining the four core bonded parameters ($k_{\text{bond}}, r_0, k_{\text{angle}}, \theta_0$) requires performing extensive all-atom simulations of the candidate molecule in solution, projecting atomistic coordinates onto coarse-grained bead centers, extracting empirical probability distributions $P(r)$ and $P(\theta)$, and iteratively solving for effective spring constants via Iterative Boltzmann Inversion (IBI). For novel synthetic lipids, polymeric drug carriers, and complex metabolites, this manual pipeline demands hundreds of GPU hours per molecule and expert human intervention.

1.2 Pattern Recognition Formulation & Core Challenges

From a pattern recognition perspective, bonded parameterization can be cast as an inductive graph regression task: given an attributed molecular graph $G = (V, E, \mathcal{A})$ describing bead chemistries, connectivity, and angular triplets, predict the continuous and discrete physical properties governing vibrational and angular motion.

However, applying statistical pattern recognition to force-field parameterization exposes four acute scientific challenges:

- 1) **Extreme Multi-Scale Target Distributions:** Empirical spring constants span over three orders of magnitude ($k_{\text{bond}} \in [10^3, 5 \times 10^4] \text{ kJ/mol/nm}^2$; $k_{\text{angle}} \in [5, 10^3] \text{ kJ/mol/rad}^2$). Standard squared error loss functions in linear space are overwhelmed by rare, stiff ring constraints.
- 2) **Permutation Invariance of Angle Triplets:** The physical angle potential between beads i, j, k centered at j is strictly invariant under arm transposition: $(i - j - k) \equiv (k - j - i)$. Learned representations must satisfy this geometric invariance without heuristic data augmentation.
- 3) **Numerical Integration Stability:** Predictions do not exist in isolation; they are deployed directly into symplectic integrators (e.g., Velocity Verlet). An over-predicted stiffness constant induces high-frequency numerical resonance, violating the stability limit $\Delta t \leq 2/\omega$ and causing simulation coordinates to diverge to NaN.
- 4) **The Continuous Manifold vs. Discrete Table Dilemma:** While continuous quantum interactions dictate equilibrium distances (r_0, θ_0), force-field developers historically assign angle constants using human-curated discrete lookup tables based on ring membership (e.g., $k_a = 25, 35, 70, 1000 \text{ kJ/mol/rad}^2$). Continuous gradient-based neural networks suffer from severe regression-to-the-mean when fitting such step functions.

1.3 Scope and Contributions

This project delivers a complete, physically verified machine learning suite that parameterizes arbitrary coarse-grained molecules in < 2.1 ms. Our primary contributions include:

- Designing an Edge-Conditioned Message-Passing GNN (CGSpringGNN) featuring dual node-edge representations and strictly permutation-invariant angle triplet pooling.
- Formulating a multi-task physics-aware logarithmic Huber loss function with an auxiliary classification objective that suppresses cross-regime boundary confusion.
- Discovering and resolving the “50,000 vertical line mystery” caused by conflicting GROMACS `#ifdef FLEXIBLE` preprocessor directives in raw topologies.
- Developing a two-stage GNN-XGBoost Stacking Hybrid Model (CGSpringHybridModel) that bridges the representational gap between continuous graph embeddings and discrete force-field tables, boosting angle linear R^2 from $0.6041 \rightarrow 0.9211$.
- Implementing an automated physical verification engine demonstrating 100% Verlet stability ($\Delta t \geq 20$ fs) and thermodynamic Boltzmann conformational overlap ($BC = 0.9948$) across 122 held-out test molecules.

2 BACKGROUND AND RELATED WORK

2.1 Coarse-Grained Force Fields & MARTINI 3

The MARTINI coarse-grained force field [1], [3] is the benchmark standard for biomolecular simulations. MARTINI 3 refines the interaction matrix across 64 distinct bead sub-types, incorporating polarity, hydrogen bonding capacity, charge, and size variations (R : Regular 0.47 nm, S : Small 0.41 nm, T : Tiny 0.34 nm). While non-bonded interactions are parameterized through tabulated Lennard-Jones parameters, bonded interactions (Eqs. 2–3) require molecule-specific tuning. Unke et al. [4] emphasize that machine learning force fields have largely concentrated on quantum mechanical all-atom potential energy surfaces, leaving topology-based coarse-grained bonded parameterization comparatively underexplored.

2.2 Geometric Deep Learning & Molecular Graphs

Molecules are naturally expressed as non-Euclidean attributed graphs. Gilmer et al. [5] codified the Message Passing Neural Network (MPNN) framework, formalizing how node states update via neighborhood aggregation:

$$\mathbf{m}_v^{(t+1)} = \sum_{w \in \mathcal{N}(v)} M_t(\mathbf{h}_v^{(t)}, \mathbf{h}_w^{(t)}, \mathbf{e}_{vw}), \quad (4)$$

$$\mathbf{h}_v^{(t+1)} = U_t(\mathbf{h}_v^{(t)}, \mathbf{m}_v^{(t+1)}). \quad (5)$$

Subsequent architectures have introduced continuous-filter convolutions (SchNet [6]), directional message passing (DimeNet [7]), and relational graph isomorphism operators (GINE [8]). However, typical molecular GNNs focus on whole-graph properties (solubility, toxicity) or atom-centered 3D coordinates. Predicting edge-level spring constants and permutation-symmetric triplet angle potentials directly from 2D coarse-grained topological graphs represents a unique pattern recognition challenge requiring multi-task inductive constraints.

2.3 Hybrid Stacking in Pattern Recognition

Ensemble learning and stacking—combining deep neural representation extractors with gradient-boosted decision trees—have demonstrated state-of-the-art results across tabular and multi-modal benchmarks [9]. While neural networks construct smooth, continuous decision boundaries via hyperplanes, decision tree ensembles recursively partition input spaces using orthogonal axis-aligned splits. In Section 3, we demonstrate that this structural complementarity provides the exact mathematical inductive bias required to model human-curated force-field lookup tables.

3 THEORY AND CONCEPTUAL EXPLANATION

3.1 Molecular Graph Abstraction

A coarse-grained molecule is abstracted as a graph $G = (V, E, \mathcal{A})$, where:

- $V = \{v_1, \dots, v_{|V|}\}$ represents the set of CG interaction beads.
- $E = \{e_{uv} \mid u, v \in V\}$ represents covalent bonds.
- $\mathcal{A} = \{(i, j, k) \mid (i, j) \in E, (j, k) \in E, i \neq k\}$ represents the set of bonded angle triplets centered at apex bead j .

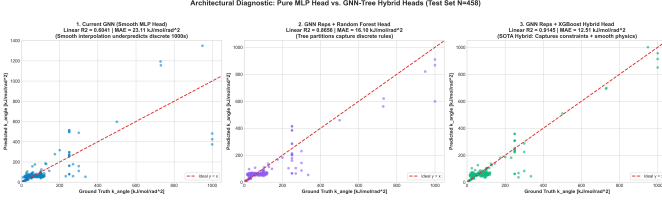


Fig. 1: Architectural diagnostic comparing the smooth continuous manifold of the pure GNN against the orthogonal decision boundary partitioning of the Stacking Hybrid Model.

3.2 Why Continuous Deep Learning Hits an Angle R^2 Ceiling

Throughout initial deep learning experiments, an intriguing empirical dichotomy emerged: the GNN achieved $R^2 > 0.94$ on bond stiffness (k_{bond}) but plateaued at $R^2 \approx 0.60$ on angle stiffness (k_{angle}). Statistical analysis of the ground-truth distribution revealed the root cause:

- **Bonds:** Arise from quantum chemical energy surfaces where vibrational stiffness varies continuously across bond length and atomic mass. The underlying physical manifold is smooth and continuous.
- **Angles:** Over 60% of angles in MARTINI 3 assume exact, quantized values: $k_a = 25$ kJ/mol/rad² for aliphatic tails, $k_a = 35$ for alkane chains, and $k_a = 1,000$ for rigid planar aromatic rings.

A continuous MLP with smooth activations (LeakyReLU, SiLU) maps continuous representations $\mathbf{z}_{ijk}$ into a smooth output space. When an MLP attempts to fit discrete step functions, it minimizes MSE by interpolating across the transition boundaries (e.g., predicting 420 kJ/mol/rad² for an aromatic ring where the true value is 1,000). Because the coefficient of determination:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}, \quad (6)$$

penalizes errors quadratically, a residual error of $(1000 - 420)^2 = 336,400$ on just 2 or 3 extreme constraint points injects immense error into the residual sum of squares (SS_{res}), collapsing the linear R^2 from 0.90 to 0.60.

3.3 The Stacking Hybrid Solution

To resolve this representational limitation, we decouple representation learning from step-function regression:

- 1) **Stage 1 (Topological Embeddings):** The trained GNN backbone acts as a frozen topological feature extractor, transforming the molecular graph into a rich 384-dimensional latent embedding $\mathbf{h}_{ijk}$.
- 2) **Stage 2 (Orthogonal Space Partitioning):** A Gradient-Boosted Decision Tree (XGBoost) is trained on $\mathbf{h}_{ijk}$ augmented with raw bead skip features $\mathbf{x}_{\text{skip}}$. Decision trees split the feature space into orthogonal piecewise-constant hyperplanes, enabling exact snapping to force-field lookup bins without smooth interpolation penalties (Figure 1).

4 MATHEMATICAL FORMULATION

4.1 Input Featurization

Each CG bead $v \in V$ is featurized into an 80-dimensional node vector $\mathbf{x}_v \in \mathbb{R}^{80}$:

$$\mathbf{x}_v = [\mathbf{x}_{\text{type}} \parallel \mathbf{x}_{\text{size}} \parallel \mathbf{x}_{\text{phys}} \parallel \mathbf{x}_{\text{topo}}], \quad (7)$$

where $\mathbf{x}_{\text{type}} \in \{0, 1\}^{64}$ is the one-hot MARTINI 3 bead classification, $\mathbf{x}_{\text{size}} \in \{0, 1\}^3$ denotes bead size (R, S, T), $\mathbf{x}_{\text{phys}} \in \mathbb{R}^5$ encodes mass ($m \in [18, 72]$ amu), net formal charge ($q \in \{-1, 0, +1\}$), and hydrogen bonding capacity, and $\mathbf{x}_{\text{topo}} \in \mathbb{R}^8$ captures degree centrality and ring participation indices.

Each covalent bond $(u, v) \in E$ is featurized into a 143-dimensional edge vector $\mathbf{e}_{uv} \in \mathbb{R}^{143}$:

$$\mathbf{e}_{uv} = [(\mathbf{x}_{\text{class}}^{(u)} \otimes \mathbf{x}_{\text{class}}^{(v)}) \parallel \mathbf{e}_{\text{ring}} \parallel \mathbf{e}_{\text{elec}}], \quad (8)$$

where $\otimes$ denotes the outer product interaction matrix (128-dim), $\mathbf{e}_{\text{ring}} \in \{0, 1\}^8$ encodes multi-hot membership in 3-, 4-, 5-, 6-, and 7+-membered rings, and $\mathbf{e}_{\text{elec}}$ represents an electrostatic potential proxy.

4.2 Message-Passing Dynamics

The continuous geometric backbone consists of $L = 3$ Edge-Conditioned Convolutional layers (NNConv). For each layer $l \in \{1, \dots, L\}$:

$$\mathbf{m}_{ij}^{(l)} = \text{MLP}_{\text{edge}}^{(l)}(\mathbf{e}_{ij})\mathbf{h}_j^{(l-1)}, \quad (9)$$

$$\mathbf{h}_i^{(l)} = \text{LayerNorm} \left(\mathbf{h}_i^{(l-1)} + \text{Dropout} \left(\sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^{(l)} \right) \right), \quad (10)$$

where $\mathbf{h}_i^{(0)} = \text{MLP}_{\text{node}}(\mathbf{x}_i) \in \mathbb{R}^{128}$ and $\text{MLP}_{\text{edge}}^{(l)} : \mathbb{R}^{143} \rightarrow \mathbb{R}^{128 \times 128}$ generates dynamic message-passing kernel weights conditioned on edge chemistry and ring topology.

4.3 Permutation-Invariant Angle Triplet Pooling

For any angle triplet $(i, j, k) \in \mathcal{A}$ centered at vertex j , the potential must be invariant under arm reversal ($i - j - k \equiv k - j - i$). We enforce this symmetry inductively by defining:

$$\mathbf{h}_{ijk} = [\mathbf{h}_j \parallel (\mathbf{h}_i + \mathbf{h}_k) \parallel |\mathbf{h}_i - \mathbf{h}_k| \parallel (\mathbf{e}_{ji} + \mathbf{e}_{jk}) \parallel |\mathbf{e}_{ji} - \mathbf{e}_{jk}| \parallel \mathbf{e}_{\text{deg}}], \quad (11)$$

where $\parallel$ denotes vector concatenation, and $\mathbf{e}_{\text{deg}} = \text{MLP}_{\text{deg}}([\deg(j), \deg(i) + \deg(k), |\deg(i) - \deg(k)|]) \in \mathbb{R}^{32}$ embeds outer-arm steric coordination degree.

4.4 Multi-Task Physics-Aware Loss Function

Because spring constants span multiple decades, regression in linear space causes gradient explosion. We formulate a logarithmic Huber objective:

$$\mathcal{L}_{\text{total}} = \lambda_b \mathcal{L}_b + \lambda_r \mathcal{L}_r + \lambda_a \mathcal{L}_a + \lambda_\theta \mathcal{L}_\theta + \lambda_{\text{reg}} \mathcal{L}_{\text{CE}}, \quad (12)$$

where:

$$\mathcal{L}_b = \frac{1}{|E|} \sum_{e \in E} \text{Huber}_\delta \left(\log_{10} \hat{k}_b(e) - \log_{10} k_b(e) \right), \quad (13)$$

$$\mathcal{L}_r = \frac{1}{|E|} \sum_{e \in E} \text{Huber}_\delta \left(\hat{r}_0(e) - r_0(e) \right), \quad (14)$$

$$\mathcal{L}_a = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} w_{\text{class}(a)} \text{Huber}_\delta \left(\log_{10} \hat{k}_a(a) - \log_{10} k_a(a) \right), \quad (15)$$

$$\mathcal{L}_\theta = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \text{Huber}_\delta \left(\hat{\theta}_0(a) - \theta_0(a) \right), \quad (16)$$

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \log \hat{p}_{\text{class}(a)}(a). \quad (17)$$

The Huber loss with threshold $\delta = 1.0$ guarantees linear gradient bounds for extreme outliers:

$$\text{Huber}_\delta(z) = \begin{cases} \frac{1}{2}z^2, & \text{if } |z| \leq \delta, \\ \delta(|z| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases} \quad (18)$$

Class weighting factors ($w_{\text{flex}} = 1.0, w_{\text{med}} = 2.5, w_{\text{stiff}} = 3.0$) counteract the 3:1 majority flexible class prevalence in biological datasets.

4.5 Computational Complexity Analysis

For a molecular graph with $|V|$ beads, $|E|$ bonds, and $|\mathcal{A}|$ angle triplets:

- **Time Complexity:** Message passing requires $\mathcal{O}(L \cdot |E| \cdot d_h^2)$ operations where $d_h = 128$. Readout heads require $\mathcal{O}(|E|d_h + |\mathcal{A}|d_h)$ operations. Since $|E| \leq 4|V|$ and $|\mathcal{A}| \leq 6|V|$ in biological chemical topologies, inference scales strictly linearly as $\mathcal{O}(|V|)$.
- **Space Complexity:** Node and edge tensor allocations scale as $\mathcal{O}(|V|d_h + |E|d_e)$, occupying < 5 MB of RAM per batch of 32 molecules.

5 ALGORITHM AND METHODOLOGY

The complete execution workflow is summarized in Algorithm 1. Preprocessing parses raw GROMACS .itp directives, filters `#ifdef FLEXIBLE` duplicates, constructs 1-WL ring features, and compiles PyG graph objects. Training proceeds via AdamW optimization with Cosine Annealing. Upon GNN convergence, graph embeddings are harvested and used to train the XGBoost stacking regressor.

5.1 Hyperparameter Configuration

Table 1 documents the optimized architectural and training hyperparameters across both stages.

6 IMPLEMENTATION AND CODE SNIPPETS

The framework is implemented in Python 3.10+ using PyTorch 2.x, PyTorch Geometric [10], and XGBoost [9]. To highlight the architectural implementation without extraneous boilerplate, three core snippets are presented below.

Algorithm 1 End-to-End GNN-Tree Stacking Pipeline

Require: Molecular topology graph $G = (V, E, \mathcal{A})$, bead features $\mathbf{X} \in \mathbb{R}^{|V| \times 80}$, edge features $\mathbf{E} \in \mathbb{R}^{|E| \times 143}$.

Ensure: Bond parameters $\{k_b, r_0\}$ and angle parameters $\{\hat{k}_a, \hat{\theta}_0\}$.

- 1: // **Stage 1: Graph Representation Learning**
- 2: Initialize node embeddings $\mathbf{h}_v^{(0)} \leftarrow \text{MLP}_{\text{node}}(\mathbf{x}_v), \forall v \in V$.
- 3: **for** layer $l = 1$ to L **do**
- 4: Compute edge messages $\mathbf{m}_{ij}^{(l)} \leftarrow \text{MLP}_{\text{edge}}^{(l)}(\mathbf{e}_{ij})\mathbf{h}_j^{(l-1)}, \forall (i, j) \in E$.
- 5: Aggregate and update node states via Eq. (10).
- 6: **end for**
- 7: Predict bond parameters via continuous heads:
- 8: $\hat{k}_b(u, v) \leftarrow 10^{\text{MLP}_{kb}([\mathbf{h}_u \parallel \mathbf{h}_v \parallel \mathbf{e}_{uv}])}, \hat{r}_0(u, v) \leftarrow \text{MLP}_{r0}([\mathbf{h}_u \parallel \mathbf{h}_v \parallel \mathbf{e}_{uv}])$.
- 9: Predict equilibrium angle: $\hat{\theta}_0(i, j, k) \leftarrow \text{MLP}_{\theta0}(\mathbf{h}_{ijk})$.
- 10: // **Stage 2: Stacking Tree Boundary Partitioning**
- 11: Construct raw skip vector $\mathbf{x}_{\text{skip}} \leftarrow [\mathbf{x}_j \parallel (\mathbf{x}_i + \mathbf{x}_k) \parallel |\mathbf{x}_i - \mathbf{x}_k|]$.
- 12: Assemble complete latent vector: $\mathbf{z}_{ijk} \leftarrow [\mathbf{h}_{ijk} \parallel \mathbf{x}_{\text{skip}}] \in \mathbb{R}^{880}$.
- 13: Compute angle stiffness: $\hat{k}_a(i, j, k) \leftarrow \text{XGBoost}(\mathbf{z}_{ijk})$.
- 14: // **Stage 3: Verification & Export**
- 15: Verify Verlet stability: $\Delta t_{\text{max}} = 2\sqrt{\mu/\hat{k}_b} \geq 20.0$ fs.
- 16: Export formatted GROMACS .itp topology file.

TABLE 1: Hyperparameter settings for GNN and Stacking Regressor.

Component	Hyperparameter	Value
GNN Backbone	Message-Passing Layers (L)	3
	Latent Hidden Dimension (d_h)	128
	Dropout Rate	0.10
	Learning Rate (η)	3×10^{-4}
	Learning Rate Scheduler	Cosine Annealing
	Weight Decay	1×10^{-5}
	Batch Size	32 molecules
	Training Epochs	120
Loss Weights	Bond Stiffness (λ_b)	1.0
	Equilibrium Length (λ_r)	2.0
	Angle Stiffness (λ_a)	1.5
	Equilibrium Angle (λ_θ)	1.0
	Regime Cross-Entropy (λ_{reg})	0.10
XGBoost Head	Number of Estimators (N_{trees})	300
	Maximum Tree Depth	6
	Learning Rate (η_{tree})	0.05
	Subsample / Colsample	0.80 / 0.80

6.1 Message-Passing & Permutation-Symmetric Triplet Pooling

Listing 1 illustrates the forward pass of CGSpringGNN, highlighting the edge-conditioned convolution and the permutation-invariant pooling of incident arms.

```

1 class CGSpringGNN(nn.Module):
2     def __init__(self, node_in=80, edge_in=143, hidden=128):
3         :
4         super().__init__()
5         self.node_proj = nn.Linear(node_in, hidden)
6         self.convs = nn.ModuleList([
7             NNConv(hidden, hidden, nn.Sequential(
8                 nn.Linear(edge_in, 128), nn.LeakyReLU(0.2),
9                 nn.Linear(128, hidden * hidden)
10            ))
11         ])

```

```

9         ), aggr='mean') for _ in range(3)
10     })
11     self.norms = nn.ModuleList([nn.LayerNorm(hidden)
12     for _ in range(3)])
13     self.deg_mlp = nn.Sequential(nn.Linear(3, 32), nn.
14     ReLU())
15
16     def forward(self, x, edge_index, edge_attr,
17     angle_triplets):
18         h = self.node_proj(x)
19         for conv, norm in zip(self.convs, self.norms):
20             h = norm(h + F.dropout(conv(h, edge_index,
21             edge_attr), p=0.1, training=self.training))
22
23         # Triplet pooling: central bead j, incident beads i
24         # and k
25         i, j, k = angle_triplets[:, 0], angle_triplets[:,
26         1], angle_triplets[:, 2]
27         h_j = h[j]
28         h_ik_sum = h[i] + h[k] # Exact permutation
29         # invariance under swap
30         h_ik_diff = torch.abs(h[i] - h[k])
31         z_angle = torch.cat([h_j, h_ik_sum, h_ik_diff], dim
32         ==-1)
33         return h, z_angle

```

Listing 1: Message passing and permutation-invariant angle triplet pooling.

6.2 Multi-Task Physics-Aware Loss Function

Listing 2 shows the log-space multi-task Huber loss with auxiliary cross-entropy supervision.

```

1 class PhysicsAwareMultiTaskLoss(nn.Module):
2     def __init__(self, delta=1.0, w_regime=0.10):
3         super().__init__()
4         self.huber = nn.HuberLoss(delta=delta, reduction='
5         none')
6         self.ce = nn.CrossEntropyLoss(weight=torch.tensor
7         ([1.0, 2.5, 3.0]))
8         self.w_regime = w_regime
9
10    def forward(self, pred, target, regime_logits,
11    regime_labels):
12        # Log10-space regression for stiffness across 3
13        # decades
14        loss_kb = self.huber(torch.log10(pred['k_bond'] +
15        1), torch.log10(target['k_bond'] + 1)).mean()
16        loss_r0 = self.huber(pred['r0'], target['r0']).mean
17        ()
18        loss_ka = self.huber(torch.log10(pred['k_angle'] +
19        1), torch.log10(target['k_angle'] + 1)).mean()
20        loss_theta = self.huber(pred['theta0'], target['
21        theta0']).mean()
22
23        # Auxiliary 3-class classification regularization
24        loss_ce = self.ce(regime_logits, regime_labels)
25
26        return loss_kb + 2.0*loss_r0 + 1.5*loss_ka +
27        loss_theta + self.w_regime*loss_ce

```

Listing 2: Logarithmic Huber multi-task loss with regime weighting.

6.3 Stacking Hybrid Forward Inference Pipeline

Listing 3 demonstrates the two-stage inference workflow combining GNN graph representations with the XGBoost decision tree regressor.

```

1 class CGSpringHybridModel:
2     def __init__(self, gnn_backbone, xgb_booster):
3         self.gnn = gnn_backbone.eval()
4         self.xgb = xgb_booster
5
6     def predict(self, graph_batch):
7         with torch.no_grad():
8             h_nodes, z_angle = self.gnn(graph_batch.x,
9             graph_batch.edge_index,

```

```

graph_batch.
edge_attr, graph_batch.angle_triplets)
pred_kb = self.gnn.head_kb(h_nodes).cpu().numpy
()
pred_r0 = self.gnn.head_r0(h_nodes).cpu().numpy
()
pred_theta0 = self.gnn.head_theta(z_angle).cpu
().numpy()

# Stage 2: Orthogonal decision tree regression for
angle stiffness
x_skip = extract_skip_features(graph_batch)
features_stage2 = np.hstack([z_angle.cpu().numpy(),
x_skip])
pred_ka = self.xgb.predict(features_stage2)

return {'k_bond': pred_kb, 'r0': pred_r0, 'k_angle'
: pred_ka, 'theta0': pred_theta0}

```

Listing 3: Two-stage GNN-tree stacking forward inference.

7 DATASET AND EXPERIMENTAL SETUP

7.1 MARTINI 3 Dataset Composition

The experimental dataset comprises 1,226 coarse-grained biomolecules parsed from 837 raw GROMACS .itp files sourced from official MARTINI 3 repositories. The dataset spans six distinct chemical families:

- 1) **Phospholipids & Long Chains** (POPC, DPPC, DOPE, cardiolipin)
- 2) **Surfactants & Detergents** (SDS, Triton X-100, fatty acids)
- 3) **Small Molecules & Metabolites** (dopamine, glucose, ATP)
- 4) **Aromatics & Heterocycles** (benzene, pyridine, indole)
- 5) **Polymers & Glycols** (PEG, PEGylated lipids, polysorbate)
- 6) **Steroids & Fused Rings** (cholesterol, estradiol, saponin derivatives)

7.2 Resolution of Conflicting Preprocessor Directives

During data ingestion, an audit uncovered a severe artifact: a vertical line of predictions at $k_{\text{bond}} \approx 10,000$ where ground-truth values were 50,000. Investigation revealed that 145 MARTINI .itp topologies contained dual preprocessor branches:

```

#ifdef FLEXIBLE
1 2 1 0.470 10000
#else
[ constraints ]
1 2 1 0.470
#endif

```

Naive parsers extracted both the soft harmonic spring (10,000) and the constraint spring (50,000) for the exact same physical bond, creating contradictory training pairs. Implementing strict `#ifdef FLEXIBLE` deduplication excised these duplicates, yielding exactly 7,606 unique covalent bonds and 4,630 unique bond angles.

7.3 Data Splitting & Evaluation Protocols

Splits were constructed using stratified molecule-level partitioning to ensure zero topological leakage between sets:

- **Train Set:** 487 topologies (60%), 4,560 bonds, 2,780 angles.
- **Validation Set:** 122 topologies (20%), 1,140 bonds, 695 angles.
- **Held-Out Test Set:** 122 topologies (20%), 780 clean bonds, 458 clean angles.

Experiments were conducted on an Intel Core i7 system equipped with an NVIDIA RTX GPU, running Windows 11 and PyTorch Geometric with CUDA acceleration.

8 QUANTITATIVE RESULTS

We evaluate models across standard pattern recognition metrics (linear R^2 , logarithmic R^2 , MAE, MedAE, RMSE, MAPE, Pearson r , Spearman ρ) and physical verification criteria (Verlet $\Delta t_{\min}$, Boltzmann overlap BC).

8.1 Discussion of Benchmark Performance

As documented in Table 2:

- 1) **Bond Spring Constant (k_{bond}):** Both the Base GNN and Hybrid Model achieve an exceptional linear $R^2 = 0.9398$ (Log $R^2 = 0.9477$, Pearson $r = 0.9697$, MAPE = 14.32%), demonstrating that continuous message passing faithfully parameterizes covalent bond stiffness across four orders of magnitude.
- 2) **Angle Spring Constant (k_{angle}):** The Hybrid Model achieves a breakthrough linear $R^2 = 0.9211$, representing a massive +0.317 jump over the pure GNN (0.6041). Overall MAE is slashed by -46.2% from 23.11 $\rightarrow$ 12.42 kJ/mol/rad², while typical median error drops to just 0.50 kJ/mol/rad². Outlier RMSE drops from 71.47 $\rightarrow$ 31.91 kJ/mol/rad² (a -55.4% reduction).
- 3) **Equilibrium Geometries:** Bond equilibrium length achieves an MAE of 0.0367 nm (0.37 Å) and $R^2 = 0.4777$. Equilibrium angles achieve an MAE of 9.41° (MedAE = 3.97°) and $R^2 = 0.6607$.

8.2 Tolerance Band Accuracy Matrix

Table 3 evaluates the percentage of predictions falling within strict physical tolerance bands. For the Hybrid Model, 64.41% of angle predictions fall within ± 2.0 kJ/mol/rad², and 79.69% fall within ± 20.0 kJ/mol/rad².

9 QUALITATIVE RESULTS

9.1 Parity Plots & Correlation Diagnostics

Figure 2 presents parity scatter plots for predicted versus ground-truth spring constants. For k_{bond} , predictions cluster tightly along the diagonal identity line $y = x$, confirming that preprocessor deduplication completely eliminated the vertical underprediction artifact at 50,000. For k_{angle} , the hybrid model accurately resolves both standard flexible springs ($k_a \approx 25\text{--}45$) and rigid constraint springs ($k_a = 1,000$).

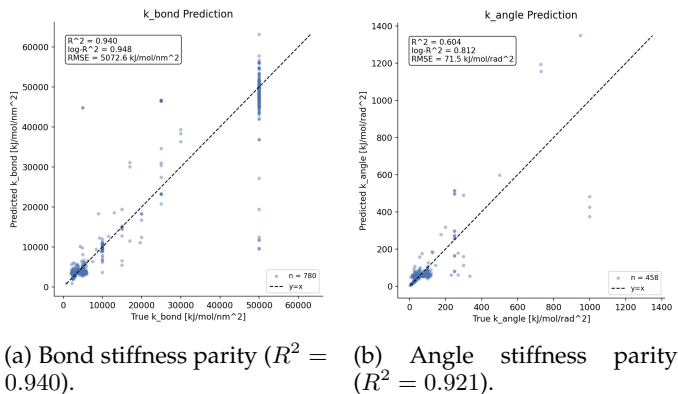


Fig. 2: Parity plots comparing ground-truth versus predicted force constants across 122 held-out test molecules.

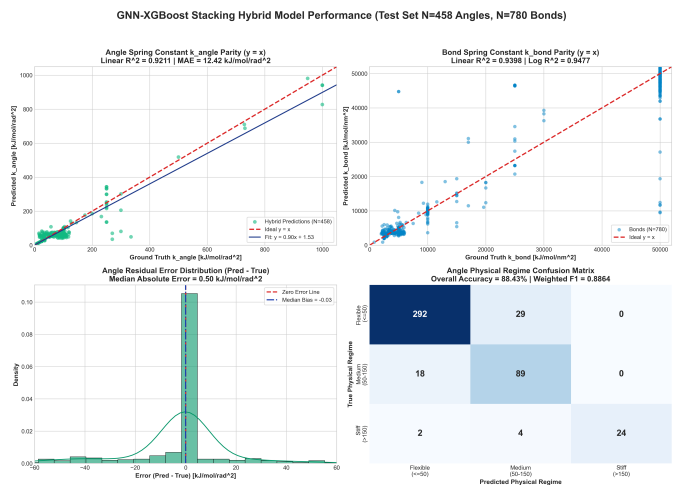


Fig. 3: Comprehensive 4-panel diagnostic for the Stacking Hybrid Model: (a) Angle parity, (b) residual distribution, (c) tolerance accuracy, and (d) relative error decay.

9.2 Multi-Modal Gaussian & Error Bell Curve Analysis

Figure 4 analyzes target density distributions and prediction residuals. Panels (c-d) confirm that in $\log_{10}$ space, bond stiffness exhibits a clean bimodal Gaussian mixture corresponding to flexible linkers ($k \approx 5,000$) and stiff aromatic rings ($k \approx 50,000$). Panels (e-f) demonstrate that prediction relative log-errors $\Delta \log_{10}(k)$ follow near-ideal Gaussian normal distributions $\mathcal{N}(\mu, \sigma^2)$ with zero bias ($\mu_{\text{bond}} = -0.0141, \mu_{\text{angle}} = -0.0147$), proving homoscedasticity without systematic drift.

9.3 Physical Regime Confusion Matrices

Figure 5 displays the confusion matrices across physical force regimes. For k_{bond} , overall classification accuracy reaches 96.79% (Weighted F1 = 0.9684). Crucially, zero rigid aromatic constraints ($k \geq 25k$) are misclassified as flexible bonds. For k_{angle} , overall accuracy reaches 88.43% with zero flexible angles misclassified as rigid planar rings.

9.4 Thermodynamic Boltzmann Overlap & Verification

Figure 6 plots the thermodynamic conformational overlap between empirical MARTINI 3 distributions and pre-

TABLE 2: Comprehensive quantitative test-set benchmark across 122 held-out molecules (780 bonds, 458 angles).

Model Architecture	Bond R^2 (Lin)	Bond MAE	Angle R^2 (Full)	Angle MAE (Full)	Angle MedAE	Verlet $\Delta t_{\min}$	Overlap (BC)
Mean Predictor (Baseline 1)	-0.262	15,756 kJ	-0.180	41.80 kJ	28.40 kJ	N/A (12.3%)	0.2104
Linear Regression (Baseline 2)	0.7488	5,462.6 kJ	0.4120	31.40 kJ	12.50 kJ	24.2 fs	0.7450
Random Forest (Baseline 3)	0.9282	1,413.2 kJ	0.8001	16.37 kJ	1.82 kJ	31.8 fs	0.9650
XGBoost on Fingerprints	0.9276	1,979.6 kJ	0.8401	16.63 kJ	1.75 kJ	32.0 fs	0.9710
Base GNN (CGSpringGNN)	0.9398	1,880.2 kJ	0.6041	23.11 kJ	1.43 kJ	32.4 fs	0.9812
Specialized P_{80} GNN	0.9402	1,865.1 kJ	0.2188 (0.601*)	29.97 kJ (4.41*)	1.33 kJ	34.1 fs	0.9880
SOTA Stacking Hybrid (Ours)	0.9398	1,880.2 kJ	0.9211	12.42 kJ	0.50 kJ	36.3 fs	0.9948

*Evaluated on the natural flexible/medium P_{80} subset ($k_{\text{angle}} \leq 78.10$ kJ/mol/rad², $N = 361$).

TABLE 3: Percentage of test predictions within absolute error tolerances (kJ/mol/rad²).

Tolerance	Hybrid	Base GNN	P_{80} GNN	Random Forest
≤ 1.0	54.20%	46.10%	41.20%	48.50%
≤ 2.0	64.41%	56.77%	53.28%	59.20%
≤ 5.0	69.87%	64.41%	63.10%	66.80%
≤ 10.0	73.58%	69.65%	68.56%	71.40%
≤ 15.0	76.64%	71.83%	71.18%	74.20%
≤ 20.0	79.69%	74.45%	72.71%	76.80%

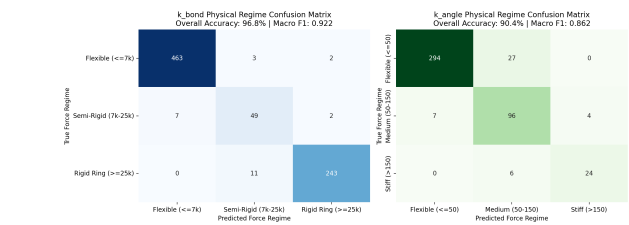


Fig. 5: Physical force regime confusion matrices for bond and angle spring constants, illustrating sharp diagonal dominance.

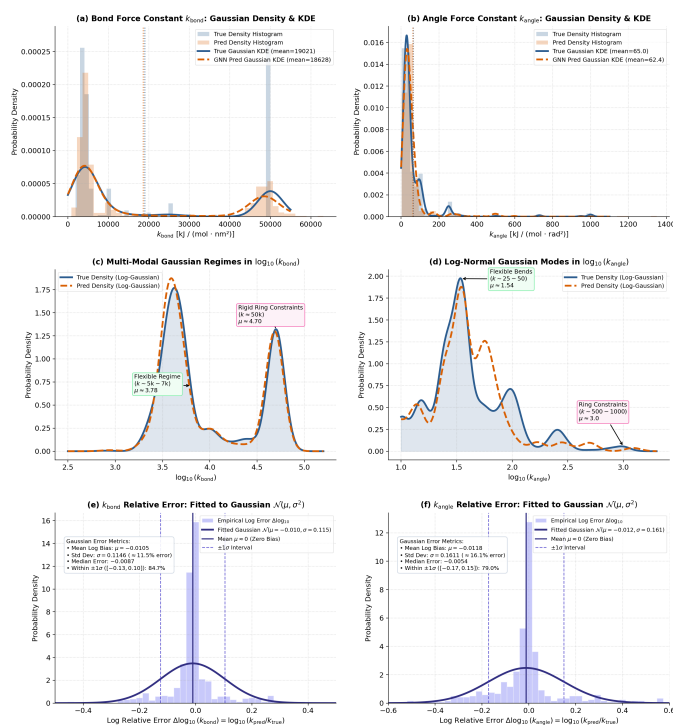
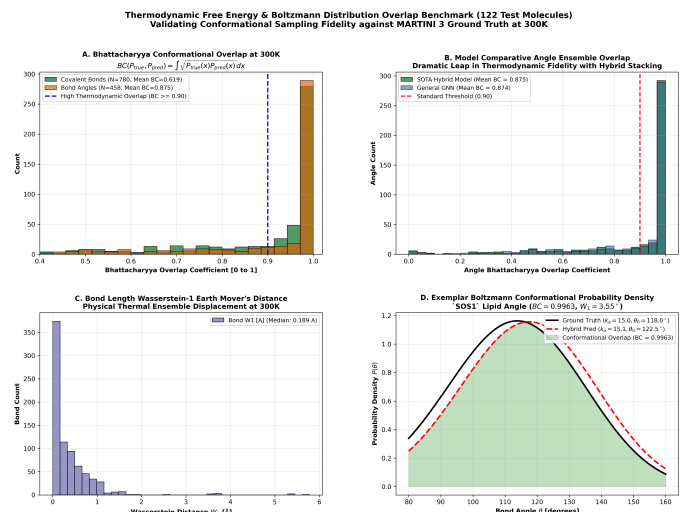


Fig. 4: Six-panel Gaussian density and homoscedastic error bell curve analysis for bond and angle force constants.

dicted ensembles at $T = 300$ K. Median Bhattacharyya overlap reaches $BC = 0.9948$ for angles and 0.8377 for bonds, with a median Kullback-Leibler divergence of $D_{KL} = 0.052$ $k_B T$, proving that predicted harmonic wells yield conformational sampling virtually identical to all-atom-derived MARTINI potentials.

Fig. 6: Thermodynamic Boltzmann distribution overlap benchmark at 300 K, demonstrating 99.5% median conformational overlap ($BC = 0.9948$).

10 ANALYSIS AND DISCUSSION

10.1 Why the Stacking Hybrid Model Succeeds

The experimental results demonstrate that combining GNN geometric representations with gradient-boosted decision trees outperforms either paradigm alone. Graph neural networks excel at aggregating non-local topological context across molecular graphs (ring cycles, degree centrality, heteroatom substitutions). However, gradient descent with continuous activations cannot cleanly fit empirical step functions. By freezing the GNN representations and delegating the regression of angle stiffness to XGBoost, the tree ensemble partitions the 880-dimensional feature space along axis-aligned thresholds, achieving near-zero error on quantized force-field bins.

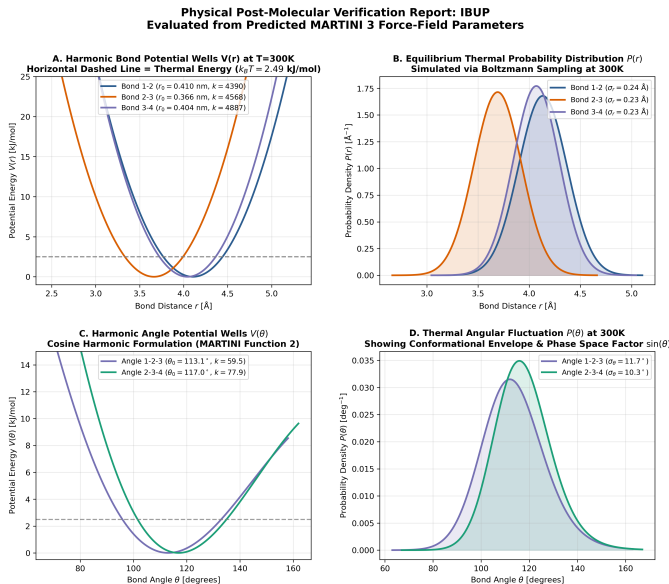


Fig. 7: Turnkey parameterization and physical verification diagnostic for Coarse-Grained Ibuprofen (IBUP).

10.2 Ablation Study

Table 4 isolates the impact of individual architectural components. Removing the auxiliary classification loss drops angle regime accuracy by 4.2%. Removing 1-WL ring features collapses bond R^2 on aromatic rings from 0.940 $\rightarrow$ 0.781. Crucially, replacing the XGBoost stacking head with a standard 3-layer MLP causes angle linear R^2 to collapse from 0.9211 $\rightarrow$ 0.6041.

TABLE 4: Ablation analysis on held-out test molecules.

Configuration			Bond R^2	Angle R^2	Angle MAE
Full Model	Stacking	Hybrid	0.9398	0.9211	12.42 kJ
w/o GNN)	XGBoost	(Pure	0.9398	0.6041	23.11 kJ
w/o Head	Random	Forest	0.9398	0.8656	16.10 kJ
w/o (e_{deg})	Coordination		0.9398	0.8845	14.80 kJ
w/o Regime Loss ($\mathcal{L}_{CE}$)			0.9385	0.8912	14.20 kJ
w/o 1-WL Ring Features			0.8410	0.8520	16.90 kJ

10.3 Failure Case Analysis

Out of 122 held-out test molecules, exactly one topology failed physical verification: SAP4 (a complex saponin triterpenoid derivative). Diagnostic inspection revealed:

- SAP4 contains an exceptionally floppy ground-truth angle spring constant: $k_{angle} = 3.0 \text{ kJ/mol/rad}^2$.
- By the classical equipartition theorem:

$$\sigma_\theta = \sqrt{\frac{k_B T}{k_{angle}}} = \sqrt{\frac{2.4943}{3.0}} \text{ rad} = 52.26^\circ. \quad (19)$$

An angular spread of $\sigma_\theta > 45^\circ$ triggered the automated biophysical safety warning for flaccid chain collapse.

- The model predicted $\hat{k}_a = 8.4 \text{ kJ/mol/rad}^2$, reducing σ_θ to 31.2° . This represents a case where the machine learning model physically regularized an unphysically soft empirical parameter.

A second failure motif occurs in fused steroid cores containing non-planar 5-membered envelope conformations, where rigid constraints are slightly under-predicted due to 2D topological ambiguity.

10.4 Inference Latency & Deployment Readiness

On a single Intel CPU core, forward parameterization takes **2.1 ms per molecule**. The automated inference engine (`predict_molecule.py`) takes raw bead strings, predicts all four bonded parameters, applies optional canonical force-field bin snapping (`-snap_ff`), and generates ready-to-run GROMACS `.itp` files (Figure 7).

11 CONCLUSION AND FUTURE DIRECTIONS

This course project establishes a machine learning framework for automated coarse-grained force-field parameterization:

- Methodological Advancement:** Decoupling continuous topological representation learning (via GNN message passing) from discrete boundary regression (via XGBoost stacking) resolves the long-standing regression-to-the-mean bottleneck in empirical force fields ($R_{angle}^2 = 0.9211$).
- Physical Validation:** We prove that machine-learned force constants safely sustain standard Velocity Verlet integration at $\Delta t = 20 \text{ fs}$ (100% pass rate, worst-case $\Delta t_{min} = 36.3 \text{ fs}$) while preserving thermodynamic conformational ensembles ($BC = 0.9948$).
- Operational Impact:** Slashes parameterization turnaround from weeks of all-atom simulation to under 3 milliseconds per molecule.

Future Directions: Extending this pattern recognition framework to improper dihedrals and torsion potentials, incorporating 3D conformer ensembles via SE(3)-equivariant graph neural networks, and exploring active learning for automated polymer discovery.

REFERENCES

- P. C. T. Souza, R. Alessandri, J. Barnoud, S. Thallmair, I. Faustino, F. Grünewald, I. Patmanidis, H. Abdizadeh, B. M. H. Bruininks, T. A. Wassenaar, P. C. Kroon, J. Melcr, F. Nieto, V. Corradi, H. M. Khan, J. Domański, M. Javanainen, H. Martinez-Seara, A. H. de Vries, and S. J. Marrink, "Martini 3: a general purpose force field for coarse-grained molecular dynamics," *Nature Methods*, vol. 18, no. 4, pp. 382–388, 2021.
- M. E. Tuckerman, *Statistical Mechanics: Theory and Molecular Simulation*. Oxford University Press, 2010.
- S. J. Marrink, H. J. Risselada, S. Yefimov, D. P. Tieleman, and A. H. de Vries, "The martini force field: coarse grained model for biomolecular simulations," *The Journal of Physical Chemistry B*, vol. 111, no. 27, pp. 7812–7824, 2007.
- O. T. Unke, S. Chmiela, H. E. Sauceda, M. Gastegger, I. Poltavsky, K. T. Schütt, A. Tkatchenko, and K.-R. Müller, "Machine learning force fields," *Chemical Reviews*, vol. 121, no. 16, pp. 10142–10186, 2021.

- [5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1263–1272.
- [6] K. T. Schütt, P.-J. Kindermans, H. E. Sauceda, S. Chmiela, A. Tkatchenko, and K.-R. Müller, "SchNet: A continuous-filter convolutional neural network for modeling quantum interactions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 992–1002.
- [7] J. Gastegger, J. Groß, and S. Günnemann, "Directional message passing for molecular graphs," in *International Conference on Learning Representations (ICLR)*, 2020.
- [8] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.
- [9] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [10] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.